

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 2

Выполнил:

Пиотуховский Александр

Группа
К3441

Проверил:

Добряков Д. И.

Санкт-Петербург

2025 г.

Задача

1. Реализовать все модели данных, спроектированные в рамках ДЗ1
2. Реализовать набор из CRUD-методов для работы с моделями данных
3. Реализовать API-эндпоинт для получения пользователя по id/email

Ход работы

1. Реализация моделей данных

Для начала все сущности, спроектированные в рамках ER-диаграммы, были реализованы в виде классов валидации данных (DTO) с использованием библиотеки Pydantic. Это позволяет строго типизировать входящие и исходящие данные, а также автоматически валидировать их. На рисунке 1 представлена модель фильма, включающая валидаторы для JSON-полей и дат.

```
class FilmDTO(FilmBase):
    description: tp.Optional[str] = None
    imdb_id: tp.Optional[str] = None
    language: tp.Optional[str] = None
    budget: int
    release_date: str
    time: tp.Optional[float] = None
    genres: tp.List[tp.Mapping[str, str|int]]
    production_companies: tp.Optional[tp.List[tp.Mapping[str, str|int]]] = None
    production_countries: tp.Optional[tp.List[tp.Mapping[str, str|int]]] = None

    class Config:
        fields = {"tagline": {"exclude": True}}

    @validator("genres", "production_companies", "production_countries", pre=True)
    def validate_json(cls, value: tp.List[tp.Mapping[str, str]] | None):
        if value is not None:
            return json.loads(value)

    @validator("release_date", pre=True)
    def validate_release_date(cls, value: date):
        return str(value)
```

Рисунок 1 – Модель данных “Фильм”

С помощью наследования от BaseModel описывается схема данных. Механизм валидаторов (декоратор @validator) используется для проверки корректности форматов (например, даты релиза) и структуры сложных JSON-полей (жанры, страны производства), что обеспечивает целостность данных на уровне приложения.

2. Реализация CRUD методов.

Для работы с созданными моделями были реализованы CRUD-методы в рамках архитектурного слоя контроллеров. Реализация выполнена на базе микрофреймворка Starlette с использованием классов HTTPEndpoint. На рисунке 2 изображен контроллер для работы с сущностью «Фильм», реализующий методы получения, создания, обновления и удаления.

```
class Film(HTTPEndpoint): 1 usage (1 dynamic)
    __service: IFilmService = container.resolve(IFilmService)

    async def get(self, request: Request):
        dto = GetFilmsDTO(**request.query_params)
        films = await self.__service.get_films_assortment(dto)

        return JSONResponse(status_code=status.HTTP_200_OK, content=films.dict())

    @requires(scopes: "authenticated", status_code=401)
    @requires(scopes: "admin", status_code=403)
    async def post(self, request: Request):
        data = await request.json()
        dto = CreateFilmDTO(**data)
        created_film = await self.__service.create_new_film(dto)

        return JSONResponse(status_code=status.HTTP_201_CREATED, content=created_film)
```

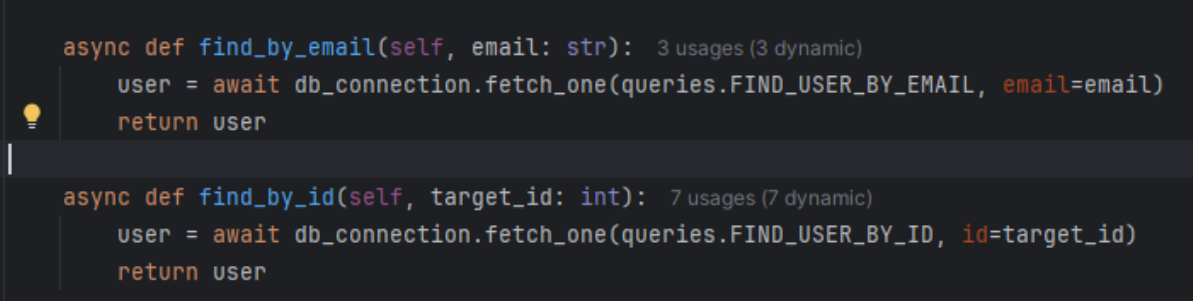
Рисунок 2 – Реализация CRUD-контроллеров для модели «Фильм»

Для каждой сущности была реализована связка «Контроллер -> Сервис -> Репозиторий». Контроллеры обрабатывают HTTP-запросы, проверяют права доступа (декоратор @requires) и вызывают методы бизнес-логики.

3. Реализация поиска пользователя по ID и Email

В качестве реализации механизма получения пользователя по идентификатору или email была разработана логика на уровне слоя доступа к данным. Поскольку проект использует «сырые» SQL-запросы

для высокой производительности, методы поиска реализованы через параметризированные запросы к базе данных PostgreSQL. На рисунке 3 представлены методы репозитория для поиска пользователя.



```
async def find_by_email(self, email: str): 3 usages (3 dynamic)
    user = await db_connection.fetch_one(queries.FIND_USER_BY_EMAIL, email=email)
    return user

async def find_by_id(self, target_id: int): 7 usages (7 dynamic)
    user = await db_connection.fetch_one(queries.FIND_USER_BY_ID, id=target_id)
    return user
```

Рисунок 3 – Методы репозитория для поиска пользователя по email или id

Данные методы используются в сервисе авторизации (для поиска по email при входе) и в сервисе профиля пользователя (для получения данных по ID), что покрывает требование к универсальному поиску пользователя.

Вывод

В ходе выполнения работы структура базы данных была перенесена в приложение с использованием DTO-моделей Pydantic. Была выстроена многослойная архитектура, в рамках которой реализованы полный набор CRUD-операций для основных сущностей и механизмы выборки пользователей по ключевым полям. Реализация выполнена на языке Python с применением асинхронного подхода.